MEENAKSHI COLLEGE OF ENGINEERING

Department of Computer Science and Engineering

CS25C09 — JAVA PROGRAMMING

PROJECT-BASED LEARNING REPORT

INVENTORY AND BILLING (POS) SYSTEM

A Java Swing and JDBC application for product stock management and point-of-sale billing

Submitted by

Name	Register No.	Role
Anand.S	311425104007	Database and JDBC
Anandhan.P	311425104008	GUI and testing
Jeevash.L	311425104039	Product and billing module
Kishore.R	311425104046	Inventory management module

Course Instructor

[Faculty Name], [Designation]

Academic Year 2026–2027 | Semester III

October 2026

CERTIFICATE

This is to certify that the project report titled “Inventory and Billing (POS) System” is a bonafide record of the work carried out by the students listed below in partial fulfilment of the requirements of the course CS25C09 Java Programming during the academic year 2026–2027.

- Anand.S - 311425104007
- Anandhan.P - 311425104008
- Jeevash.L - 311425104039
- Kishore R - 311225104046

_____ _____

Faculty In-charge Head of the Department

[Name], [Designation] [Name], [Designation]

Place: [Place] Date: [DD Month YYYY]

Submitted for the project viva voce examination held on [DD Month YYYY].

DECLARATION

We hereby declare that the project report titled “Inventory and Billing (POS) System” is our original work, carried out under the guidance of [Faculty Name]. The report has not been submitted to any other course or institution. Wherever the work of others has been used, it has been acknowledged in the references.

Place: [Place] Kishore.R, Jeevash.L, Anand.S and Anandhan.P

Date: [DD Month YYYY]

ACKNOWLEDGEMENT

We express our sincere gratitude to [Faculty Name], [Designation], Department of Computer Science and Engineering, for the guidance, encouragement and feedback provided throughout this project. We thank the Head of the Department and the management of Meenakshi College of Engineering for providing the facilities required to develop and test the application. We also thank our classmates for reviewing the application and suggesting improvements, and our families for their support.

ABSTRACT

Small retail shops often manage product stock and customer bills manually or with separate records. This can cause calculation errors, inaccurate stock counts, slow checkout and difficulty reviewing previous sales. The Inventory and Billing (POS) System is a desktop application designed using Java to combine product management, stock tracking and point-of-sale billing in one interface. The application uses object-oriented classes to represent products, cart items and sales. A Java Swing interface provides forms, tables and menu actions for common operations. JDBC connects the application to a relational database so product details and completed sales can be stored persistently. Java collections manage the active shopping cart, while exception handling validates inputs and reports problems such as insufficient stock or invalid quantities. File I/O can be used to export a customer receipt as a text file. The design separates the user interface, business logic and database access to improve readability and maintenance. Functional test cases are defined for adding products, searching inventory, billing, stock deduction, invalid input and database failure. Actual test results and screenshots should be inserted after running the implementation in the development environment.

Keywords: Java, Swing, JDBC, POS, inventory management, billing, collections, exception handling

TABLE OF CONTENTS

1 Introduction

1.1 Background

1.2 Problem Statement

1.3 Objectives

1.4 Scope and Limitations

1.5 Syllabus Units Covered

2 Requirement Analysis

2.1 Functional Requirements

2.2 Non-Functional Requirements

2.3 Software and Hardware Requirements

3 System Design

3.1 Architecture

3.2 Package Structure

3.3 Class Design

3.4 Database Design

4 Implementation

4.1 OOP Model and Product Management

4.2 Swing UI and Billing Workflow

4.3 JDBC and Transaction Handling

4.4 Collections and Search

4.5 Receipt File Export

4.6 Exception Handling and Challenges

5 Testing and Results

5.1 Test Strategy

5.2 Test Cases

5.3 Exception Handling Summary

5.4 Results and Discussion

6 Conclusion and Future Work

References

Appendix A: Database Script

Appendix B: How to Build and Run

Appendix C: Project Evaluation Sheet

1. Introduction

1.1 Background

Retail stores need to maintain product names, prices, quantities and sales records. During checkout, a cashier must identify each product, calculate the amount, apply any permitted discount or tax, collect payment and update stock. When these activities are performed manually, the process can become slow and errors may occur.

A Point-of-Sale (POS) application combines checkout and inventory operations. This project demonstrates how Java can be used to develop a small desktop solution suitable for a demonstration retail shop. It focuses on core Java programming concepts rather than advanced commercial features.

1.2 Problem Statement

A manual inventory and billing process may lead to incorrect totals, duplicate records, stock quantities that do not match actual sales, and difficulty retrieving earlier transactions. The goal is to build a Java application that maintains product records, prepares bills from a cart, checks stock availability, records completed sales and generates a readable receipt.

1.3 Objectives

1. Create, view, update and search product records using a structured interface.
2. Maintain stock quantity and unit price for each product.
3. Add products to a cart and calculate subtotal, discount, tax and final amount accurately.
4. Prevent billing when requested quantities exceed available stock.
5. Store products and completed sales in a relational database through JDBC.
6. Generate a receipt file and show clear messages for invalid input or failures.

1.4 Scope and Limitations

In scope:

- Product registration and editing
- Stock quantity tracking
- Product search by name or product code
- Cart management and bill calculation

- Sales record storage and receipt export
- Input validation and error handling

Out of scope:

- Online payments and payment gateway integration
- Multi-branch cloud synchronization
- Barcode scanner integration
- GST filing or legally certified accounting reports
- User roles and remote access unless added as an extension

Assumptions: the application is used on one cashier workstation, the database is available locally or on the same network, and tax/discount percentages are configurable demonstration values. Replace these assumptions with the rules specified by the instructor.

1.5 Syllabus Units Covered

Unit	Java concepts	Application in this project
I	Classes, objects, constructors, encapsulation, inheritance, static members	Product, CartItem, Sale and ID generation
II	Packages, interfaces, exceptions, generics, lambda expressions, multithreading	Layered packages, custom stock exception, typed collections; optional background task
III	String handling, utility classes, event handling, layout managers	Product search, Swing form events and layout
IV	Console/file I/O, Swing, menus	Receipt export, menu bar, dialogs and tables
V	JDBC API, connections, statements, database operations	Saving products and sales using prepared statements and transactions
VI	Collections, regex, arrays, Map, iterators	Cart list, product filtering, sorting and lookup maps

2 Requirement Analysis

2.1 Functional Requirements

ID	Requirement	Priority
FR-01	The cashier can add, view, edit and remove product records.	High
FR-02	The system stores product code, name, unit price and stock quantity.	High
FR-03	The cashier can search products by code or name.	High
FR-04	The cashier can add and remove products from the billing cart and change quantities.	High
FR-05	The system calculates subtotal, discount, tax and final total.	High
FR-06	The system prevents a sale when the requested quantity exceeds available stock.	High
FR-07	After a successful sale, the system records the bill and reduces stock in the database.	High
FR-08	The system can save or export a receipt to a text file.	Medium
FR-09	The system displays validation messages for invalid input and database errors.	High
FR-10	The cashier can view previous sales using a transaction identifier or date.	Medium

2.2 Non-Functional Requirements

- Usability: common cashier operations should be accessible from clearly labelled controls.
- Accuracy: monetary calculations should use BigDecimal rather than binary floating-point arithmetic.
- Reliability: stock reduction and sale recording should be performed in one database transaction.
- Performance: product search should remain responsive for the demonstration dataset.
- Maintainability: UI, model, service and DAO code should be separated into packages.
- Security: SQL statements should use PreparedStatement; credentials should not be hard-coded in source code.

2.3 Software and Hardware Requirements

Item	Specification
Programming language	Java (JDK 21 or later recommended)
GUI	Java Swing
Database	MySQL 8.x
Database connectivity	MySQL Connector/J JDBC driver
IDE	IntelliJ IDEA, Eclipse or VS Code
Operating system	Windows, Linux or macOS with a compatible JDK
Hardware	A computer with 4 GB RAM or more and sufficient free disk space

3 System Design

3.1 Architecture

The system uses a layered architecture. The Swing UI collects input and displays product and billing data. The service layer applies validation and billing rules. The DAO layer performs database operations through JDBC. The MySQL database stores product and sales information.

```
Swing UI (MainFrame, ProductPanel, BillingPanel)
                    |
                    v
Service Layer (InventoryService, BillingService)
                    |
                    v
DAO Layer (ProductDAO, SaleDAO, SaleItemDAO, DBUtil)
                    |
                    v
MySQL Database (products, sales, sale_items)
```

Figure 3.1: Layered architecture of the Inventory and Billing (POS) System.

3.2 Package Structure

```
com.college.pos
  model      Product, CartItem, Sale, SaleItem
  dao        ProductDAO, SaleDAO, SaleItemDAO, DBUtil
  service    InventoryService, BillingService, ReceiptService
  exception  InsufficientStockException, InvalidProductException
  ui         MainFrame, ProductPanel, BillingPanel, SalesPanel
  util       CurrencyUtil, InputValidator, ReceiptWriter
```

3.3 Class Design

Class	Type	Responsibility
Product	Class	Stores product ID/code, name, price and stock quantity.
CartItem	Class	Stores a product and the quantity selected for billing.
Sale	Class	Represents a completed transaction and its totals.
SaleItem	Class	Stores a product, quantity and unit price captured at sale time.
InventoryService	Class	Validates product data and stock changes.
BillingService	Class	Calculates totals and coordinates the checkout transaction.
ProductDAO	Class	Performs product CRUD and search operations using JDBC.
SaleDAO	Class	Creates and retrieves sales records.
InsufficientStockException	Custom exception	Reports when the requested quantity exceeds stock.

3.4 Database Design

Table	Important columns	Keys / purpose
products	product_id, product_code, name, unit_price, stock_qty	Primary key product_id; unique product_code
sales	sale_id, sale_date, subtotal, discount, tax, total, payment_method	Primary key sale_id; one row per completed bill
sale_items	sale_item_id, sale_id, product_id, quantity, unit_price, line_total	Foreign keys to sales and products; one row per billed item

The sale_items table stores the unit price used at the time of sale. This preserves the historical bill even if the product price changes later.

4 Implementation

The following code excerpts illustrate the intended implementation. They are reference snippets for the report and must be integrated, compiled and tested in the complete project before claiming successful execution.

4.1 OOP Model and Product Management (Units I and VI)

The Product class encapsulates product data using private fields. Constructors initialize objects, getters provide controlled access, and validation is performed before saving a product.

```
public class Product {
    private int productId;
    private String productCode;
    private String name;
    private BigDecimal unitPrice;
    private int stockQty;

    public Product(int id, String code, String name,
                   BigDecimal price, int stockQty) {
        this.productId = id;
        this.productCode = code;
        this.name = name;
        this.unitPrice = price;
        this.stockQty = stockQty;
    }
    public String getName() { return name; }
    public BigDecimal getUnitPrice() { return unitPrice; }
    public int getStockQty() { return stockQty; }
}
```

BigDecimal is used for currency values to avoid common floating-point rounding errors. The final application should also provide setters or update methods with validation as required.

4.2 Swing UI and Billing Workflow (Units III and IV)

The main window can use JFrame with a menu bar and tabs or panels for Products, Billing and Sales History. JTable displays products and cart lines. Buttons such as Add Product, Add to Cart, Remove Item, Checkout and Export Receipt use event listeners. UI creation should start on the Event Dispatch Thread.

```
SwingUtilities.invokeLater(() -> {
    JFrame frame = new JFrame("Inventory and Billing POS");
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setSize(1000, 650);
    frame.setLocationRelativeTo(null);
    frame.setContentPane(new MainFramePanel());
    frame.setVisible(true);
});
```

Figure 4.1: Insert a screenshot of the main POS window here.

Figure 4.2: Insert a screenshot of the billing cart and checkout summary here.

4.3 JDBC and Transaction Handling (Unit V)

PreparedStatement is used to safely insert product details. In checkout, the sale, sale items and stock changes should be committed together. If any operation fails, the transaction should be rolled back to avoid partial updates.

```
String sql = "INSERT INTO products"
    + "(product_code, name, unit_price, stock_qty)"
    + " VALUES (?, ?, ?, ?)";
try (Connection con = DBUtil.getConnection();
     PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setString(1, product.getProductCode());
    ps.setString(2, product.getName());
    ps.setBigDecimal(3, product.getUnitPrice());
    ps.setInt(4, product.getStockQty());
    ps.executeUpdate();
}
```

For checkout, disable auto-commit, verify stock, insert the sale and sale items, update quantities, then commit. On SQLException or a stock validation failure, roll back and show an understandable message.

4.4 Collections and Search (Units II, III and VI)

An ArrayList<CartItem> can hold the current cart. A product search can normalize input with trim() and compare names without case sensitivity. A Comparator can sort product results by name or code.

```
List<Product> results = products.stream()
    .filter(p -> p.getName().toLowerCase()
        .contains(query.trim().toLowerCase())
        || p.getProductCode().equalsIgnoreCase(query.trim()))
    .sorted(Comparator.comparing(Product::getName))
    .toList();
```

4.5 Receipt File Export (Unit IV)

After a sale is committed, ReceiptWriter can write the bill number, date, product lines, quantities, prices, tax, discount and final amount to a text file. The application should catch IOException and inform the cashier if the receipt cannot be written.

```
try (BufferedWriter writer = Files.newBufferedWriter(path)) {
    writer.write("INVENTORY AND BILLING - RECEIPT");
    writer.newLine();
    writer.write("Bill No: " + sale.getSaleId());
    writer.newLine();
    writer.write("Total: Rs. " + sale.getTotal());
} catch (IOException ex) {
    JOptionPane.showMessageDialog(null,
        "Receipt could not be saved: " + ex.getMessage());
}
```

4.6 Exception Handling and Challenges

Challenge	Solution
Requested quantity is higher than stock	Validate available stock before checkout and throw InsufficientStockException.
Invalid price or quantity input	Validate numeric input and require non-negative prices and positive quantities.
Sale saved but stock not updated	Use a single JDBC transaction for sale rows and stock updates.
Duplicate product code	Enforce a UNIQUE constraint in the database and show a friendly message.
Receipt export fails	Catch IOException and keep the completed sale record intact.
Slow database operation freezes UI	Use a SwingWorker for long-running database operations and update UI on the EDT.

5 Testing and Results

5.1 Test Strategy

Testing should cover valid operations, boundary values and failure cases. Each requirement should be tested with a known input and an observable expected result. Run tests against a test database rather than live store data. Replace the expected/actual status below with results from the completed application.

5.2 Test Cases

TC	Scenario	Input	Expected result	Status
TC-01	Add valid product	Code P001, price 25.00, stock 10	Product saved and shown in list	To test
TC-02	Duplicate product code	Add P001 again	Duplicate rejected with message	To test
TC-03	Search product	Search P001 or part of name	Matching product shown	To test
TC-04	Add to cart	Product P001, quantity 2	Cart line and line total displayed	To test
TC-05	Invalid quantity	Quantity 0 or negative	Input rejected	To test
TC-06	Insufficient stock	Stock 2, request quantity 3	Checkout blocked; stock unchanged	To test
TC-07	Successful checkout	Valid cart within stock	Sale and sale items saved; stock reduced	To test
TC-08	Total calculation	Two items and configured tax/ discount	Expected subtotal and final total shown	To test
TC-09	Database unavailable	Stop DB service before save	Error message shown; app does not crash	To test
TC-10	Receipt export	Export a completed sale	Receipt file created with correct total	To test

5.3 Exception Handling Summary

Exception / failure	When it occurs	Handling
InsufficientStockException	Requested quantity exceeds stock	Cancel checkout and show available quantity.
IllegalArgumentException	Invalid product fields or quantity	Validate input and display a clear message.
SQLException	Database query or update fails	Roll back transaction, log details and show a user-friendly message.
IOException	Receipt file cannot be written	Show an export error; preserve the completed sale.
NumberFormatException	Text entered in a numeric field	Reject input and request a valid number.

5.4 Results and Discussion

The application should be evaluated against the functional requirements after implementation. Record the actual test outcome and include screenshots of product management, cart calculation, successful checkout and receipt export. Do not report tests as passed until they have been run. Useful

measurements include search response time for a sample product list and checkout time for a bill with several items.

6 Conclusion and Future Work

The Inventory and Billing (POS) System is designed to demonstrate how Java programming concepts can solve a practical retail problem. The planned application combines product management, stock checking, billing calculations, JDBC persistence and receipt export in a layered desktop design. It provides practical application of object-oriented programming, collections, exception handling, Swing event handling, file I/O and database transactions. Completion should be verified through the test cases in Chapter 5.

Possible future enhancements:

- Barcode scanner support for faster product entry.
- Role-based login for cashier and administrator.
- Daily and monthly sales reports with charts.
- GST configuration and formatted invoices based on applicable requirements.
- Backup and restore for product and sales data.
- Receipt printing and support for multiple payment methods.

References

[1] Oracle, “Java Platform, Standard Edition Documentation,” https://docs.oracle.com/en/java/.

[2] Oracle, “The Java Tutorials: Creating a GUI With Swing,” https://docs.oracle.com/javase/tutorial/uiswing/.

[3] Oracle, “JDBC Basics,” https://docs.oracle.com/javase/tutorial/jdbc/basics/.

[4] MySQL, “MySQL 8.0 Reference Manual,” https://dev.mysql.com/doc/refman/8.0/en/.

[5] Anna University, Regulation 2025, CS25C09 Java Programming syllabus, as applicable to the student’s programme.

Use one consistent reference style and include only the sources actually consulted. Access dates should be added before submission.

Appendix A: Database Script

```
CREATE DATABASE IF NOT EXISTS pos_db;
USE pos_db;

CREATE TABLE products (
    product_id INT AUTO_INCREMENT PRIMARY KEY,
    product_code VARCHAR(30) NOT NULL UNIQUE,
    name VARCHAR(120) NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL CHECK (unit_price >= 0),
    stock_qty INT NOT NULL CHECK (stock_qty >= 0)
);

CREATE TABLE sales (
    sale_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    sale_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    subtotal DECIMAL(12,2) NOT NULL,
    discount DECIMAL(12,2) NOT NULL DEFAULT 0,
    tax DECIMAL(12,2) NOT NULL DEFAULT 0,
    total DECIMAL(12,2) NOT NULL,
    payment_method VARCHAR(30) NOT NULL
);

CREATE TABLE sale_items (
    sale_item_id BIGINT AUTO_INCREMENT PRIMARY KEY,
    sale_id BIGINT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL CHECK (quantity > 0),
    unit_price DECIMAL(10,2) NOT NULL,
    line_total DECIMAL(12,2) NOT NULL,
    FOREIGN KEY (sale_id) REFERENCES sales(sale_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);
```

Appendix B: How to Build and Run

7. Install a supported JDK and MySQL server.
8. Create the pos_db database and run the SQL script in Appendix A.
9. Add MySQL Connector/J to the project dependencies.
10. Configure the JDBC URL, database username and password in a local properties file; do not commit credentials.
11. Compile the Java source files and run the class containing main().
12. Test product creation, searching, cart totals, checkout, stock deduction and receipt export.
13. Add screenshots and a README containing setup steps, the Java version, database setup and known limitations.

```
# Example compile command from a simple source layout
javac -cp ".:lib/mysql-connector-j.jar" -d out $(find src -name "*.java")

# Example run command (replace with your actual main class)
java -cp "out:lib/mysql-connector-j.jar" com.college.pos.ui.Main
```

Appendix C: Project Evaluation Sheet (25 Marks)

Component	Maximum marks	Awarded	Remarks
Design and Planning	3		
Technical implementation and Java concepts	15		
Testing and Exception Handling	3		
Demo, Viva and Documentation	4		
Total	25		

Faculty signature: _____ Date: _____